SmartPG

A Cloud-Native Microservices Application

A Paying Guest accommodation management platform — built for scale, security, and resilience on Kubernetes.

M Vignesh

What is SmartPG?

SmartPG is a production-grade PG accommodation management platform — **5 backend microservices + 1 frontend**, containerized with Docker and orchestrated on Kubernetes across dev and prod environments.

Core Services

Tenant Service (4001) — JWT auth, tenant CRUD, room allocation, MongoDB

Payment Service (4002) — Payment records, status tracking, monthly billing

Frontend — React 18 + Vite + TailwindCSS, Nginx reverse proxy, role-based routing

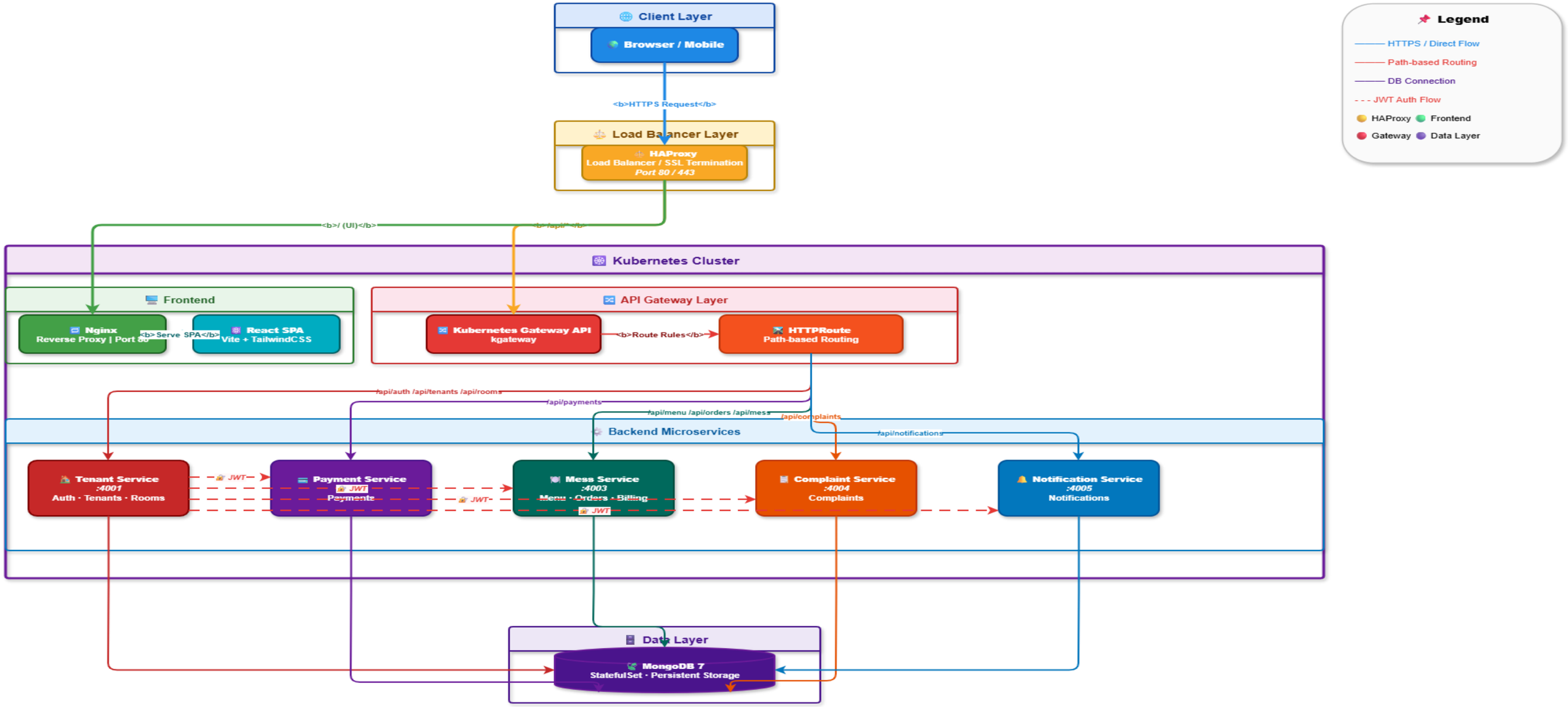
Additional Services

Complaint Service (4004) — Ticket tracking, categories, priority levels

Notification Service (4005) — Broadcast & targeted alerts, read tracking

Mess Service (4003) — Weekly menus, meal opt-in/out, rate billing

Application Flow Diagram



Key Features & Capabilities

Tenant Management

Self-registration with approval workflow, admin CRUD, RBAC

Room Management

Single/double/triple occupancy, allocate & release, occupancy tracking

Payments

Monthly billing cycles, payment summaries, status tracking

Mess & Canteen

Weekly menu CRUD, per-meal opt-in/out, automated billing

Complaints

Category & priority tracking, status lifecycle, admin resolution notes

Notifications

Targeted & broadcast messaging, read/unread tracking, multi-type alerts

Containerization with Docker

Backend Services (Multi-Stage)

- **Base Image:** node:20-alpine — minimal attack surface
- **Install:** npm install --omit=dev + COPY src/
- **Security:** adduser pgapp (UID 1001), USER pgapp
- **Serve:** EXPOSE 4001–4005, HEALTHCHECK enabled

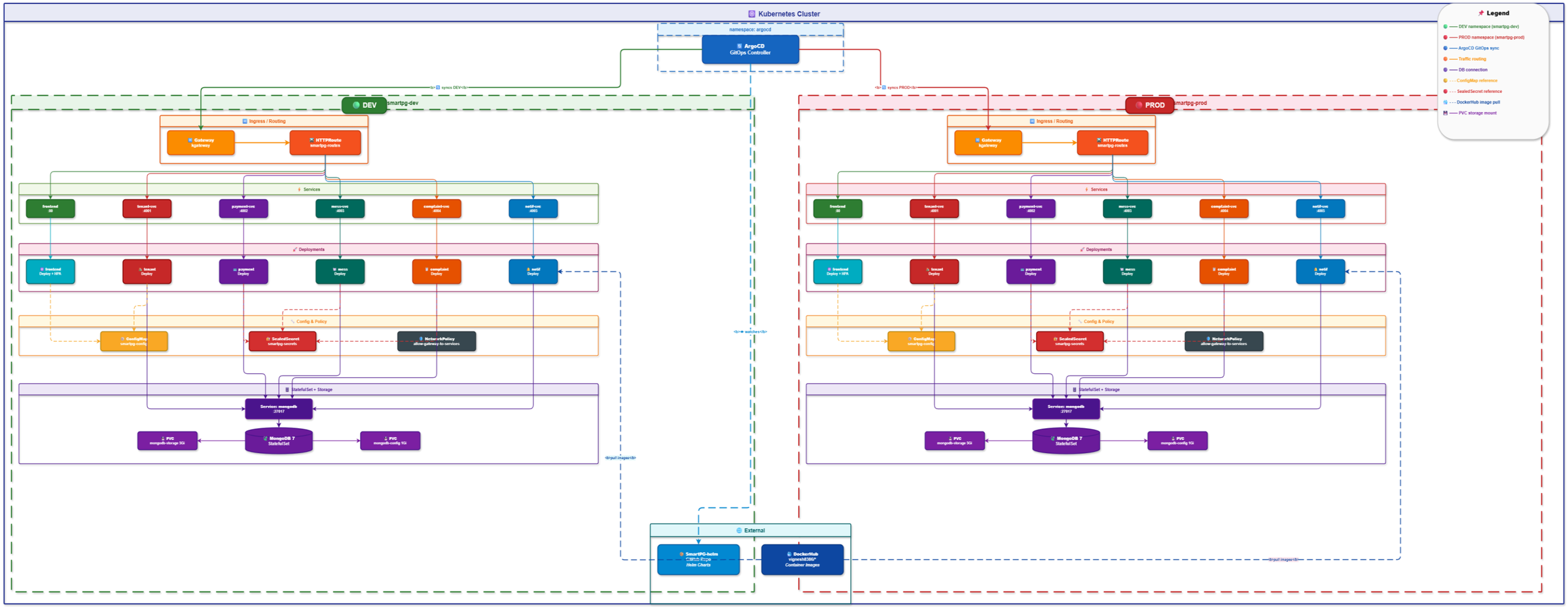
Frontend (Multi-Stage)

Stage 1 — Builder: node:20-alpine, npm install & build → /app/dist

Stage 2 — Runtime: nginx:alpine, copy dist & nginx.conf, runtime env injection via env.sh, API reverse proxy

ⓘ 📦 **Image Tagging:** Dev → Git short SHA (e.g., `a1b2c3d`) · Prod → Semantic version (e.g., `v1.2.3`) · Registry: `vignesh8386/smartpg-*`

Kubernetes Architecture



Traffic flows from the browser through the Gateway API, which uses HTTPRoute rules to route requests to the appropriate microservice by path. All services persist data to a shared MongoDB StatefulSet with 5Gi persistent volumes. Security is enforced via ConfigMaps for service discovery, SealedSecrets for encrypted credentials, and NetworkPolicies implementing zero-trust networking between pods.

CI/CD Pipeline

🌿 Dev Branch — Continuous Deployment

1. **SAST** — SonarQube scan, quality gate
2. **Build & Test** — npm install, build, unit tests
3. **SCA** — Snyk dependency scan
4. **Container Security** — Docker build (SHA tag), Trivy scan, push to DockerHub
5. **GitOps Update** — Bot updates `values-dev.yaml`, ArgoCD auto-sync

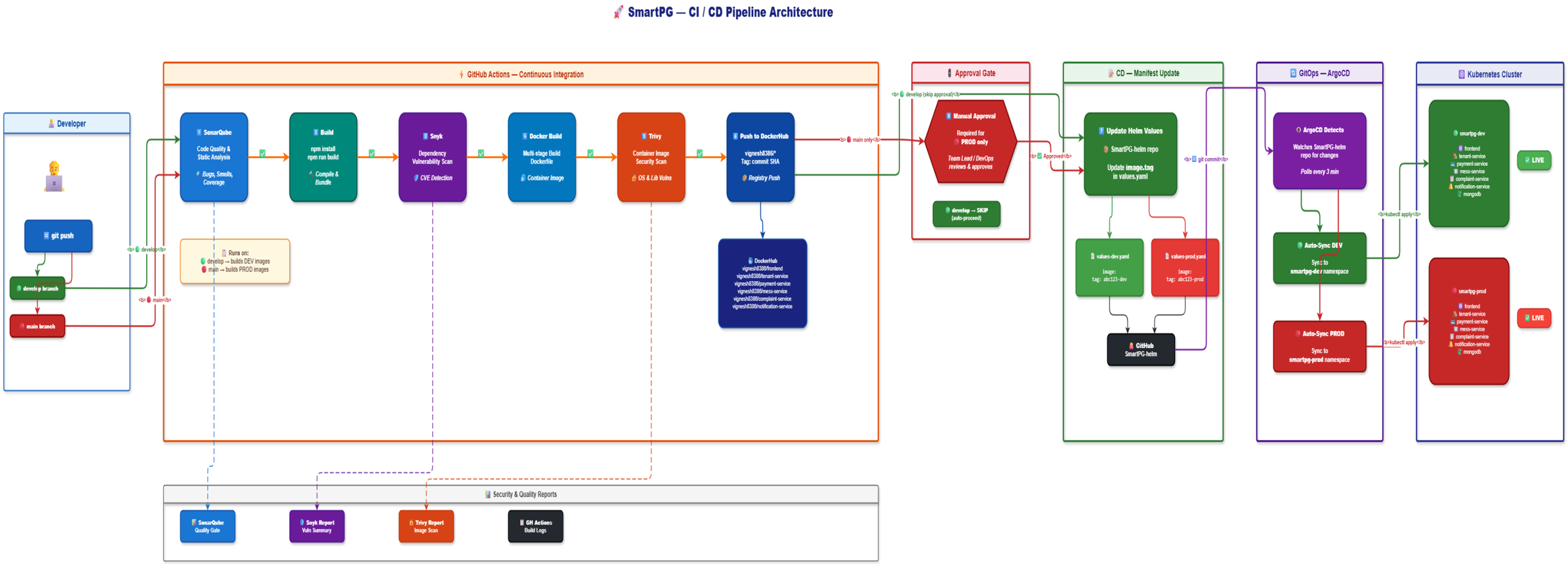
✅ **Deployed to Dev — automatic**

🚀 Main Branch — Continuous Delivery

1. **SAST** — SonarQube analysis, quality gate
2. **Build & Test** — npm install, build, test suite
3. **SCA** — Snyk scan, security validation
4. **Container Security** — Docker build (SHA tag), Trivy scan, push to DockerHub
5. **Approval Gate** 🚦 — Manual approval, email notification, GitHub Environment: production
6. **Release & Deploy** — Semantic version tag, Docker push, update `values-prod.yaml`, ArgoCD sync

✅ **Deployed to Prod — after approval**

CI/CD Pipeline Architecture



DevSecOps Security Scanning

🔍 SAST — SonarQube

Code quality, security hotspots, coverage tracking. Quality gate: **PASS required.**

🛡️ SCA — Snyk

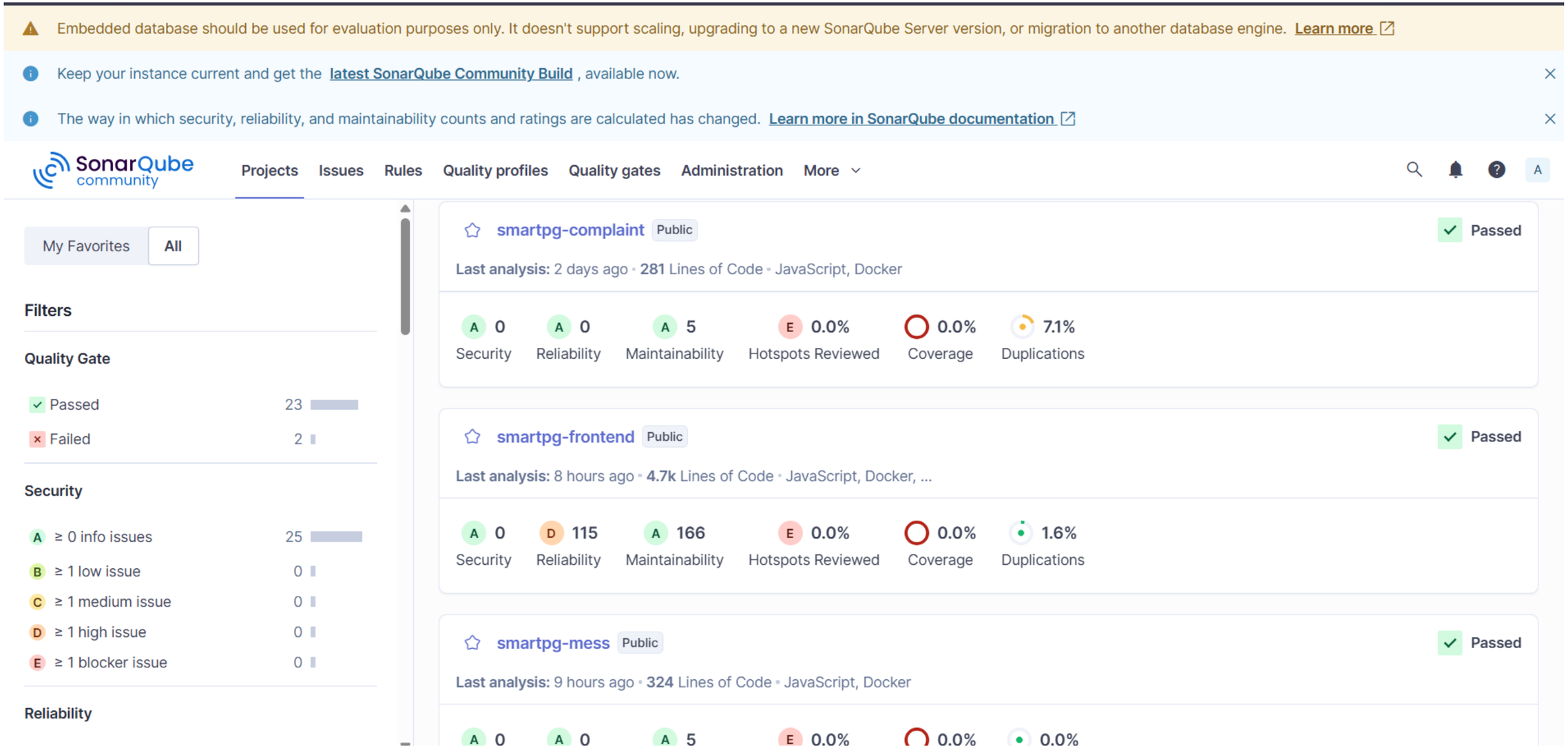
Dependency vulnerabilities, license compliance. Severity threshold: **HIGH.** Automated PR fixes.

🧪 Container — Trivy

Image & OS package scanning. Exits on **CRITICAL/HIGH.** Blocks deployment on critical findings.

✅ 🔒 **Additional Security Layers:** SealedSecrets (encrypted at rest) · NetworkPolicy (zero-trust) · Non-root containers (UID 1001) · RBAC enabled · Secret rotation support

SonarQube


Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more
Keep your instance current and get the latest SonarQube Community Build , available now.
The way in which security, reliability, and maintainability counts and ratings are calculated has changed. Learn more in SonarQube documentation
SonarQube community
Projects
Issues
Rules
Quality profiles
Quality gates
Administration
More
A
My Favorites
All
Filters
Quality Gate
Passed 23
Failed 2
Security
A ≥ 0 info issues 25
B ≥ 1 low issue 0
C ≥ 1 medium issue 0
D ≥ 1 high issue 0
E ≥ 1 blocker issue 0
Reliability
smartpg-complaint Public
Passed
Last analysis: 2 days ago • 281 Lines of Code • JavaScript, Docker
A 0 Security
A 0 Reliability
A 5 Maintainability
E 0.0% Hotspots Reviewed
0.0% Coverage
7.1% Duplications
smartpg-frontend Public
Passed
Last analysis: 8 hours ago • 4.7k Lines of Code • JavaScript, Docker, ...
A 0 Security
D 115 Reliability
A 166 Maintainability
E 0.0% Hotspots Reviewed
0.0% Coverage
1.6% Duplications
smartpg-mess Public
Passed
Last analysis: 9 hours ago • 324 Lines of Code • JavaScript, Docker
A 0
A 0
A 5
E 0.0%
0.0%
0.0%

Synk Test Report

snyk

Snyk test report

April 28th 2026, 12:45:19 am (UTC+00:00)

Scanned the following path:

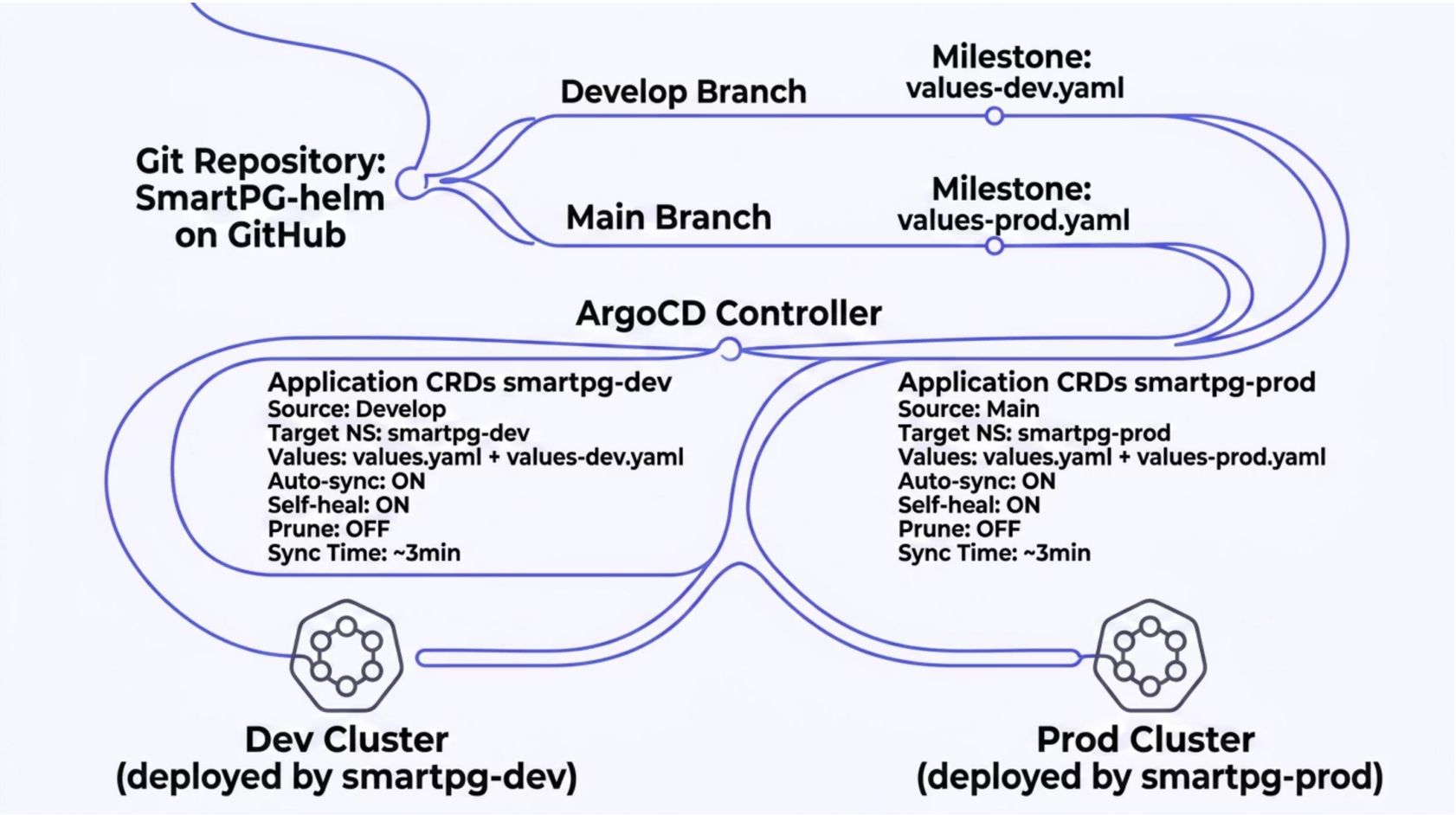
- /home/runner/work/frontend/frontend/package-lock.json (npm)

0 known vulnerabilities | 0 vulnerable dependency paths | 31 dependencies

Project	frontend	Path	/home/runner/work/frontend/frontend
Package Manager	npm	Manifest	package-lock.json

No known vulnerabilities detected.

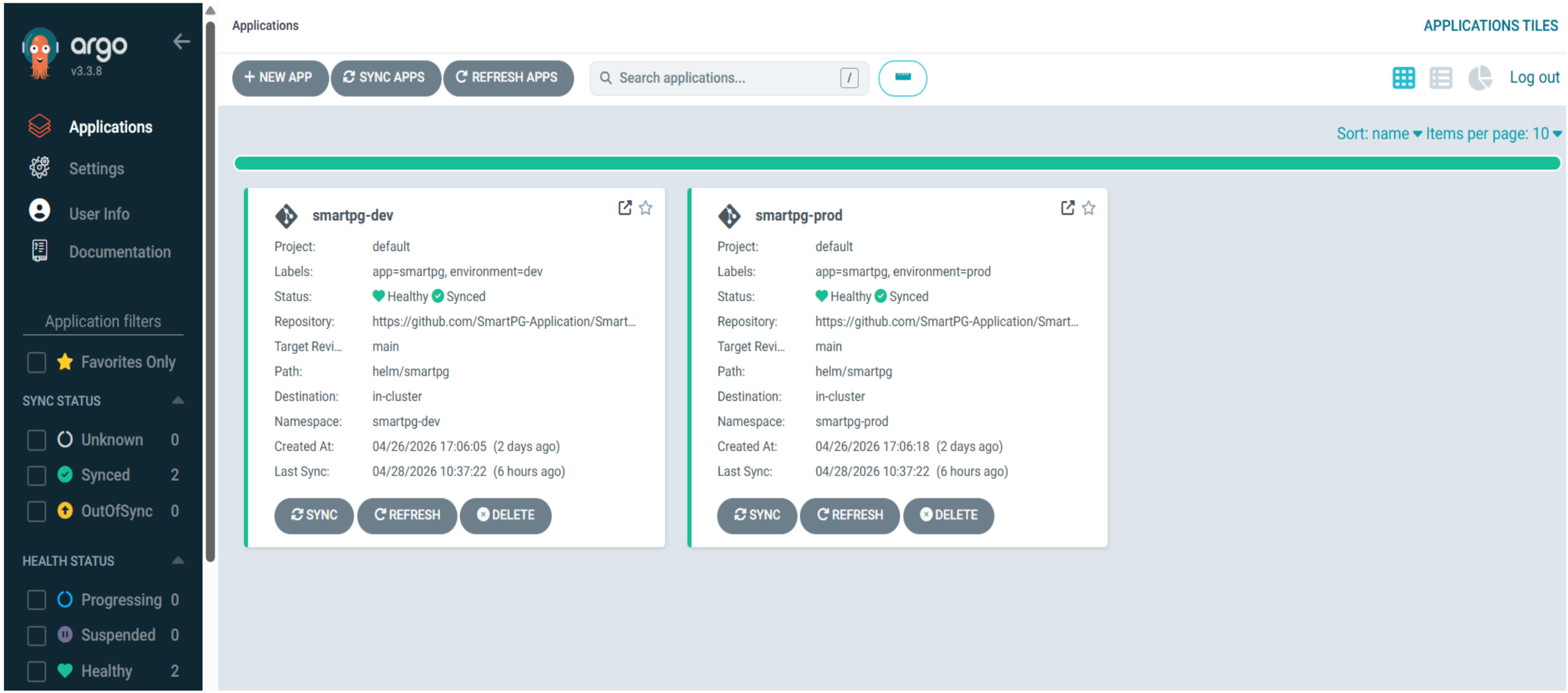
GitOps with ArgoCD



Key Features highlighted:

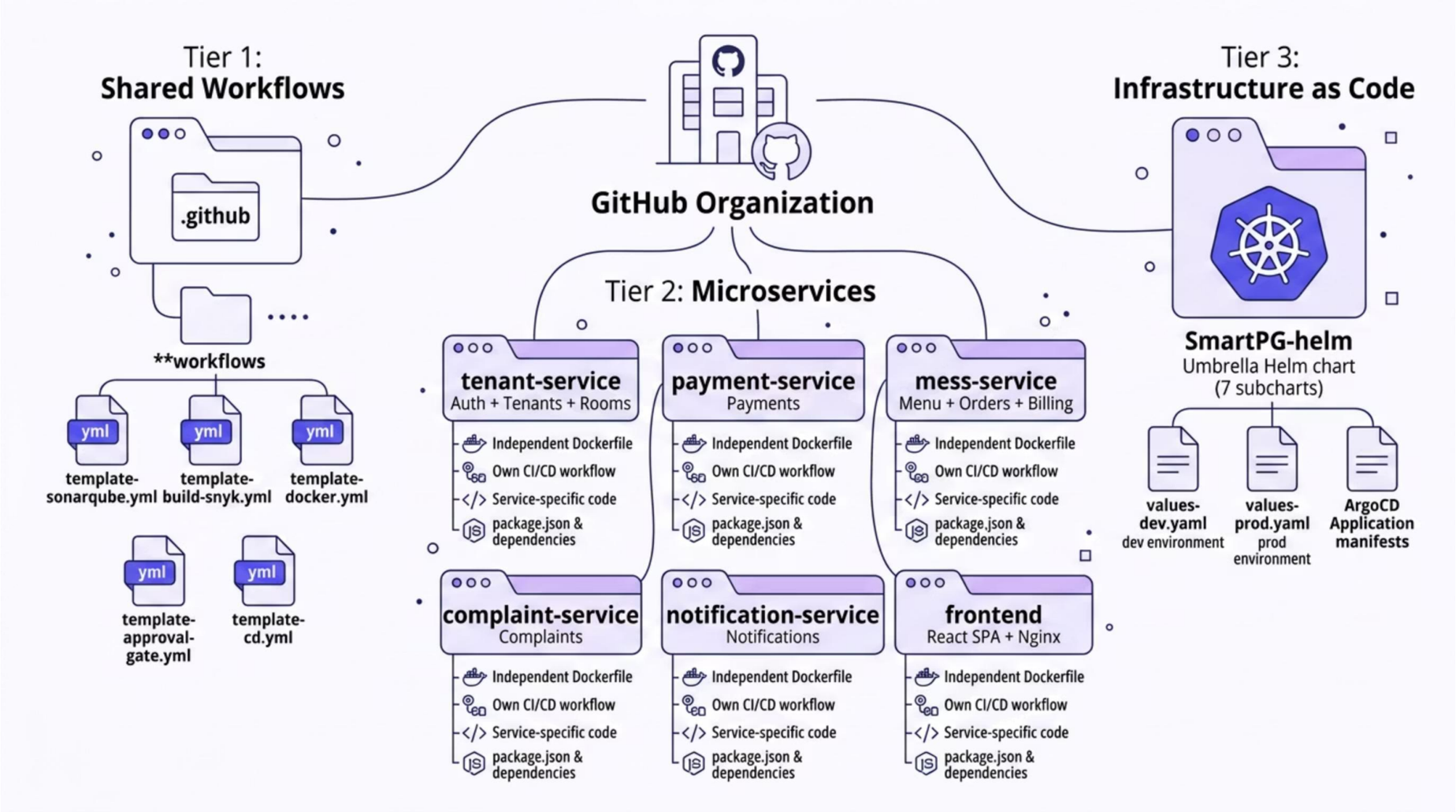
- ✅ Declarative GitOps
- ✅ Automated sync
- ✅ Drift detection
- ✅ Rollback capability
- ✅ Health monitoring

ArgoCD


argo
v3.3.8
Applications
Settings
User Info
Documentation
Application filters
Favorites Only
SYNC STATUS
Unknown 0
Synced 2
OutOfSync 0
HEALTH STATUS
Progressing 0
Suspended 0
Healthy 2
Applications
APPLICATIONS TILES
+ NEW APP
SYNC APPS
REFRESH APPS
Search applications...
Log out
Sort: name
Items per page: 10
smartpg-dev
Project: default
Labels: app=smartpg, environment=dev
Status: Healthy Synced
Repository: https://github.com/SmartPG-Application/Smart...
Target Revi... main
Path: helm/smartpg
Destination: in-cluster
Namespace: smartpg-dev
Created At: 04/26/2026 17:06:05 (2 days ago)
Last Sync: 04/28/2026 10:37:22 (6 hours ago)
SYNC
REFRESH
DELETE
smartpg-prod
Project: default
Labels: app=smartpg, environment=prod
Status: Healthy Synced
Repository: https://github.com/SmartPG-Application/Smart...
Target Revi... main
Path: helm/smartpg
Destination: in-cluster
Namespace: smartpg-prod
Created At: 04/26/2026 17:06:18 (2 days ago)
Last Sync: 04/28/2026 10:37:22 (6 hours ago)
SYNC
REFRESH
DELETE

GitHub Organization Structure

SmartPG-Application | 8 Repositories | Full Microservices Architecture



Monitoring - Grafana

Thank You

GitHub Organization link : https://github.com/SmartPG-Application

SmartPG Application: http://smartpg.online/login

Default Credentials (Demo)

- Admin: admin@smartpg.com / Admin@123
- Tenant: rahul@smartpg.com / Tenant@123